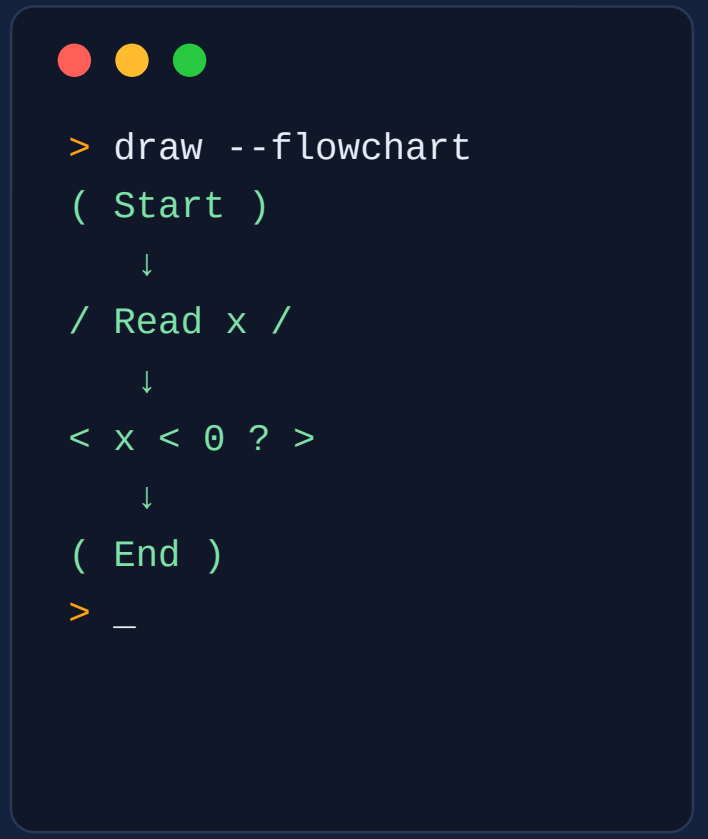
>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 2

Flowcharts and Their Symbols



Basic flowchart examples · from pictures to pseudocode to a first look at C#

Asst. Prof. Hasan Oğuz, PhD

Istanbul Okan University · Vocational School

2026–2027 Fall Semester

By the End of Today You Can …

Name the symbols

Recognise and draw the standard flowchart symbols (ISO 5807) and say what each one means.

Follow the rules

Apply the drawing rules: one Start, one End, two labelled exits per decision, arrows everywhere.

Spot the structures

Recognise sequence, selection and repetition as flowchart patterns.

Convert both ways

Turn pseudocode into a flowchart and a flowchart into pseudocode.

Trace a flowchart

Execute a flowchart by hand with a trace table and predict its output.

Map to C# (preview)

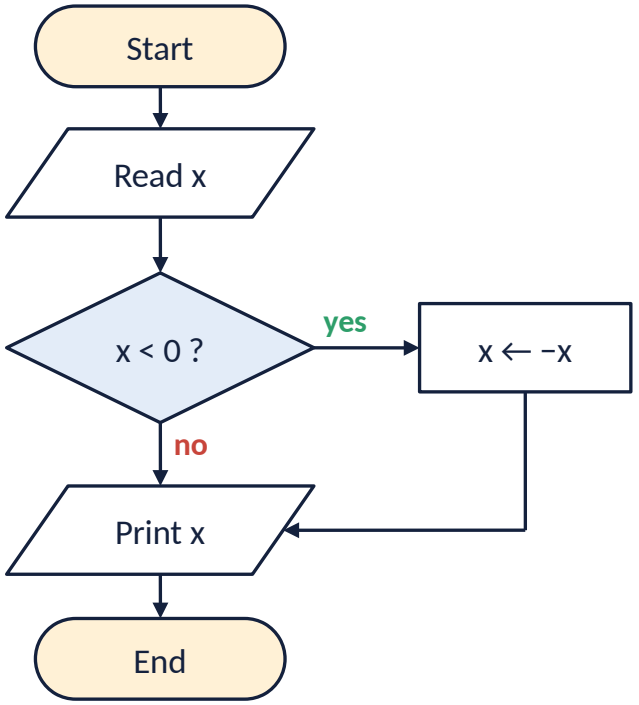
See how each box becomes one or two lines of a C# program.

One Algorithm, Three Notations

Week 1 · pseudocode

```
START
  READ x
  IF x < 0 THEN
    x ← −x
  END IF
  PRINT x
END
```

Week 2 · flowchart



Week 3+ · C# (02_02_AbsoluteValue)

```
Console.Write("Enter a number: ");
double x = double.Parse(Console.ReadLine());

if (x < 0)
{
    x = -x;
}

Console.WriteLine($"Absolute value: {x}");
```

Same steps, different clothes. Pseudocode is quick to write, a flowchart shows the paths at a glance, C# can be run.

PART 1

Flowchart Symbols

Six shapes, one kind of line — and the rules that make a chart readable

>_ DEFINITION

What Is a Flowchart?

Flowchart

A diagram of an algorithm: each shape is one step, and the shape tells you the kind of step. Arrows (flow lines) show the order in which the steps run.

- Visual: branches and loops are easy to see.
- Language-independent: anyone can read it.
- Exam tool: drawn by hand, like pseudocode.

A short history

- **1921** — F. & L. Gilbreth present the “flow process chart” for factory work.
- **1947** — Goldstine & von Neumann plan computer programs with “flow diagrams”.
- **1970** — ANSI X3.5 standardises the symbols in the USA.
- **1985** — ISO 5807 — the international standard we follow.
- **today** — UML activity diagrams, BPMN, Flowgorithm: same idea.

The Six Basic Symbols



Terminal

Start or End of the algorithm. One Start, one End.



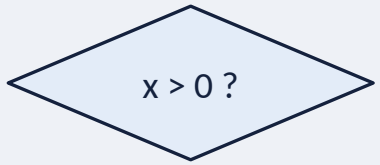
Input / Output

READ data from the keyboard or PRINT results.



Process

A calculation or assignment (←).



Decision

A yes/no question. Exactly two labelled exits.



Predefined process

A named sub-algorithm defined elsewhere.

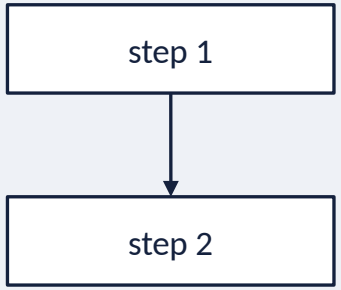


Connector

Joins flow lines or continues the chart at the same letter.

Flow Lines, Connectors and Annotations

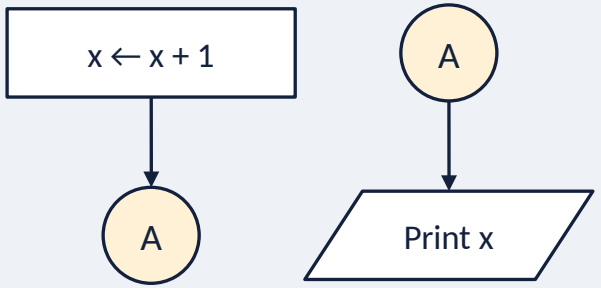
Flow lines





Top → bottom, left → right. Always draw the arrowhead: loops flow upwards.

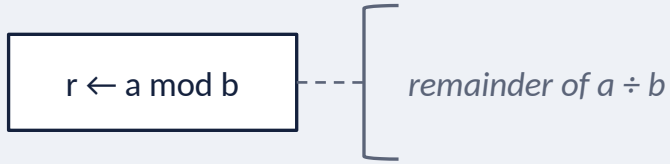
Connectors





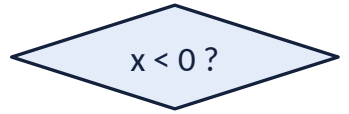
Same letter = same point. The home-plate shape continues on another page (here: page 2).

Annotation



An open bracket + dashed line adds a comment. It is not a step and never “runs” — like // comments in C#.

Every Box Has a Pseudocode Line

Symbol	Pseudocode (Week 1)	C# (from Week 3)
	`START … END`	`the program begins / ends`
	`READ x`	`x = int.Parse(Console.ReadLine());`
	`PRINT x`	`Console.WriteLine(x);`
	`x ← x + 1`	`x = x + 1;`
	`IF x < 0 THEN …  /  WHILE i ≤ N …`	`if (x < 0) …  /  while (i <= n) …`
	`use sub-algorithm GCD(a, b)`	`a method call, e.g. Math.Sqrt(x)  (Week 12)`

← assigns, = compares (pseudocode) · C#: = assigns, == compares.

>_ DRAWING RULES

Eight Rules for a Correct Flowchart

1. **One Start, one End**
   Every path from Start must reach End.

2. **Arrows on every line**
   Main direction top → bottom, left → right.

3. **Decisions: exactly two exits**
   Label them yes / no (or true / false).

4. **Boxes: one way in, one way out**
   Branches join again before continuing.

5. **No crossing lines**
   Re-route, or use a connector circle.

6. **Short, precise text**
   “sum ← sum + i”, not “add it up”.

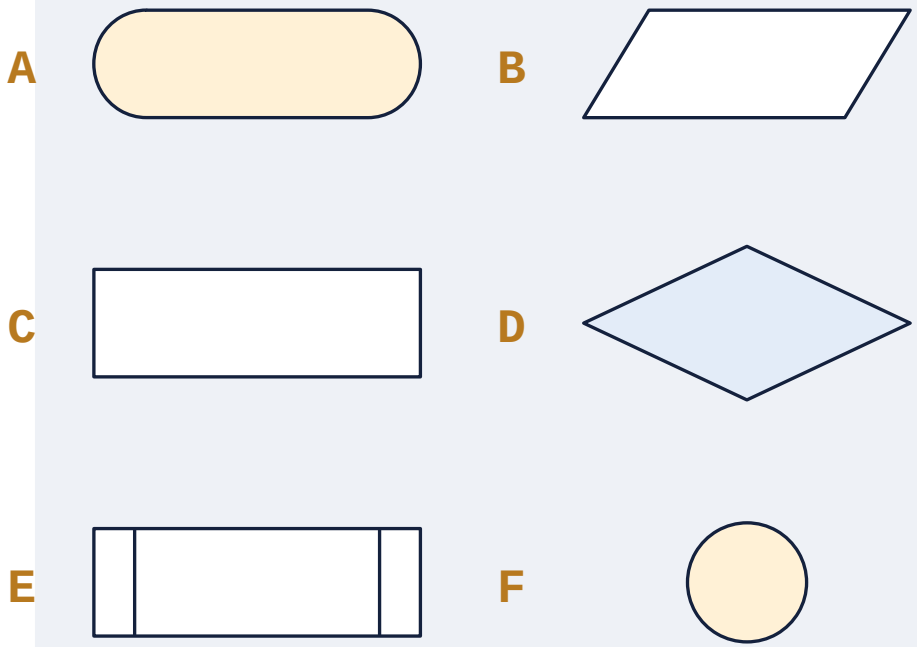
7. **Loops must make progress**
   Something the condition tests must change.

8. **Same names as the pseudocode**
   One idea per box; no mixed READ + calculate.

>_ QUICK QUIZ · 3 MIN

Which Symbol Would You Use?

#	Step
1	Read age
2	total ← total + price
3	age ≥ 18 ?
4	Print "Welcome"
5	GCD(a, b) — described on page 2
6	Start
7	Read x, then x ← 2 × x



Match each step 1–7 to a shape A–F.

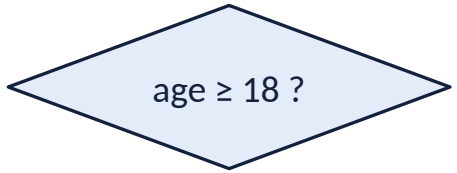
Each Step in Its Shape



1 · input / output



2 · process



3 · decision



4 · input / output

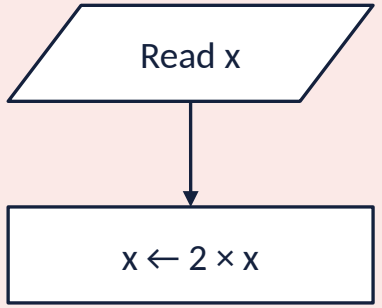


5 · predefined process



6 · terminal

7 · trap!



two boxes

One idea per box. Item 5 (GCD) uses the predefined-process shape because the details are written elsewhere; its “page 2” chart would start with its own Start.



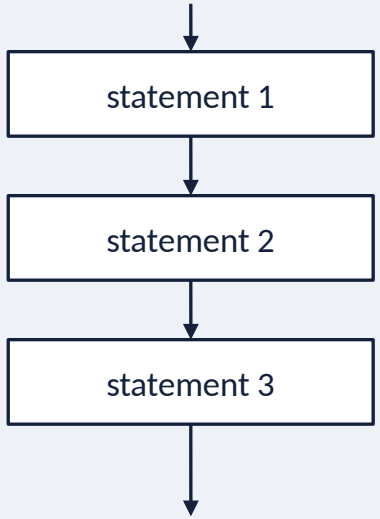
PART 2

Three Structures as Flowchart Patterns

Sequence, selection, repetition — and converting between charts and pseudocode

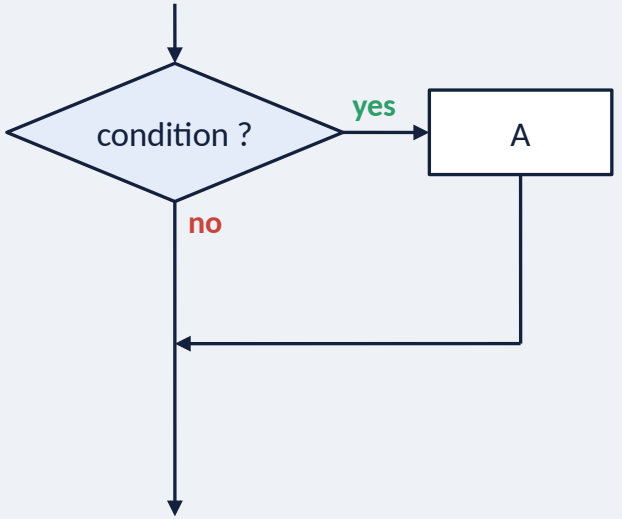
Sequence and Selection

Sequence



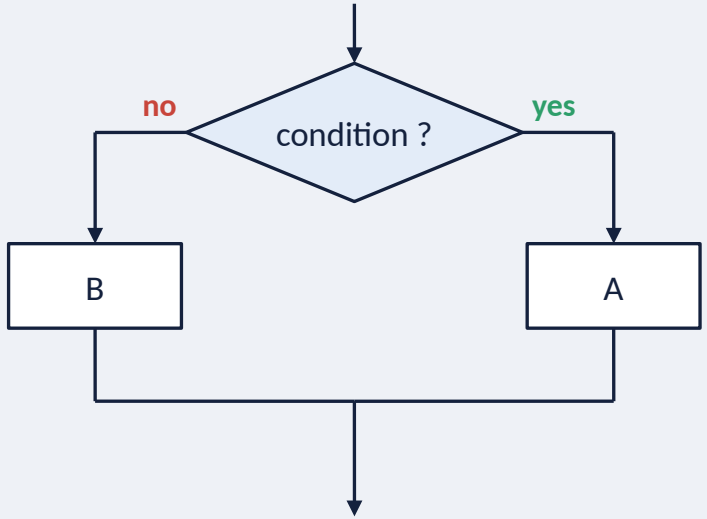

```
statement 1
statement 2
statement 3
```

Selection · IF–THEN




```
IF condition THEN
   A
END IF
```

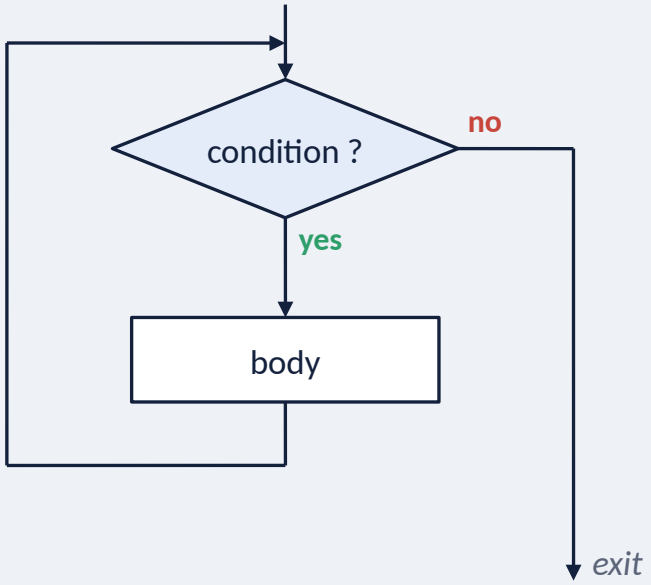
Selection · IF–THEN–ELSE




```
IF condition THEN
   A
ELSE
   B
END IF
```

Repetition: the Arrow That Goes Back Up

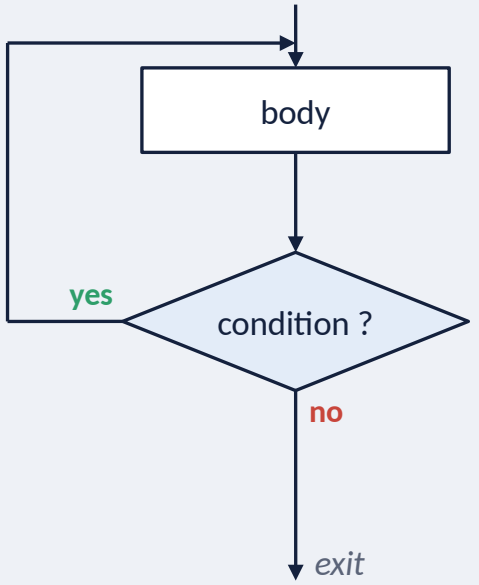
Pre-test loop · WHILE




```
WHILE cond
    body
END WHILE
```

Condition first: the body may run 0 times.

Post-test loop · preview (Week 7)




```
do
{
    body
} while (cond);
```

Body first: runs at least once.

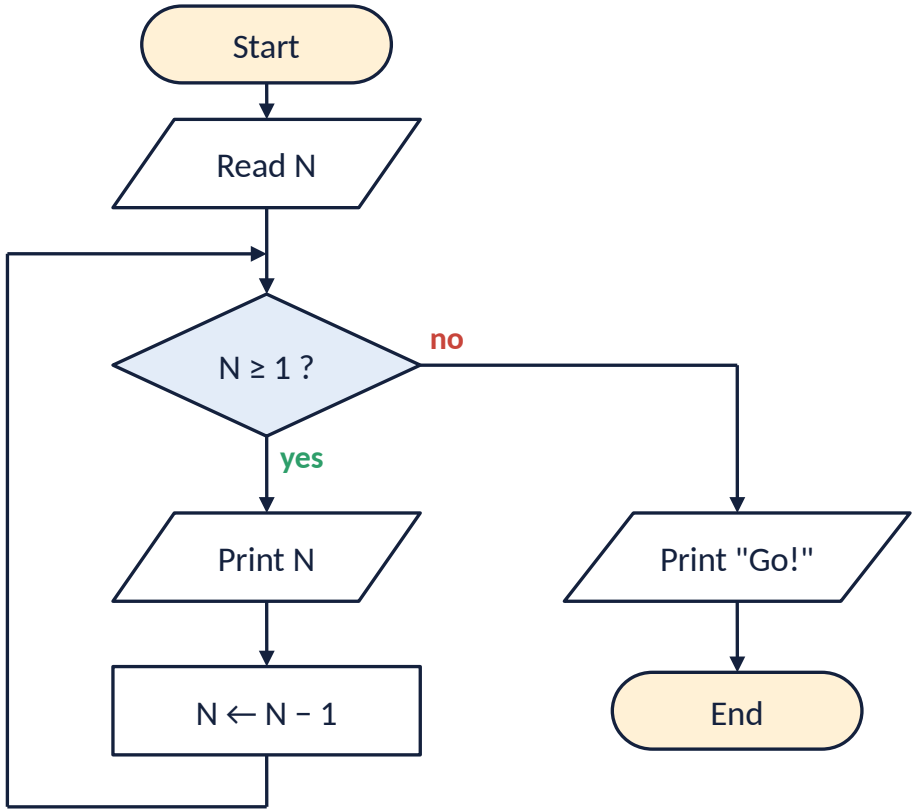
An arrow pointing upwards = repetition. Something inside the body must change the condition, or the loop never ends.

Countdown: One Line, One Box

Week 1 · exercise D

```
START
  READ N
  WHILE N ≥ 1
    PRINT N
    N ← N − 1
  END WHILE
  PRINT "Go!"
END
```

Trace N = 3 → 3, 2, 1, Go!
Edge case N = 0 → only “Go!”



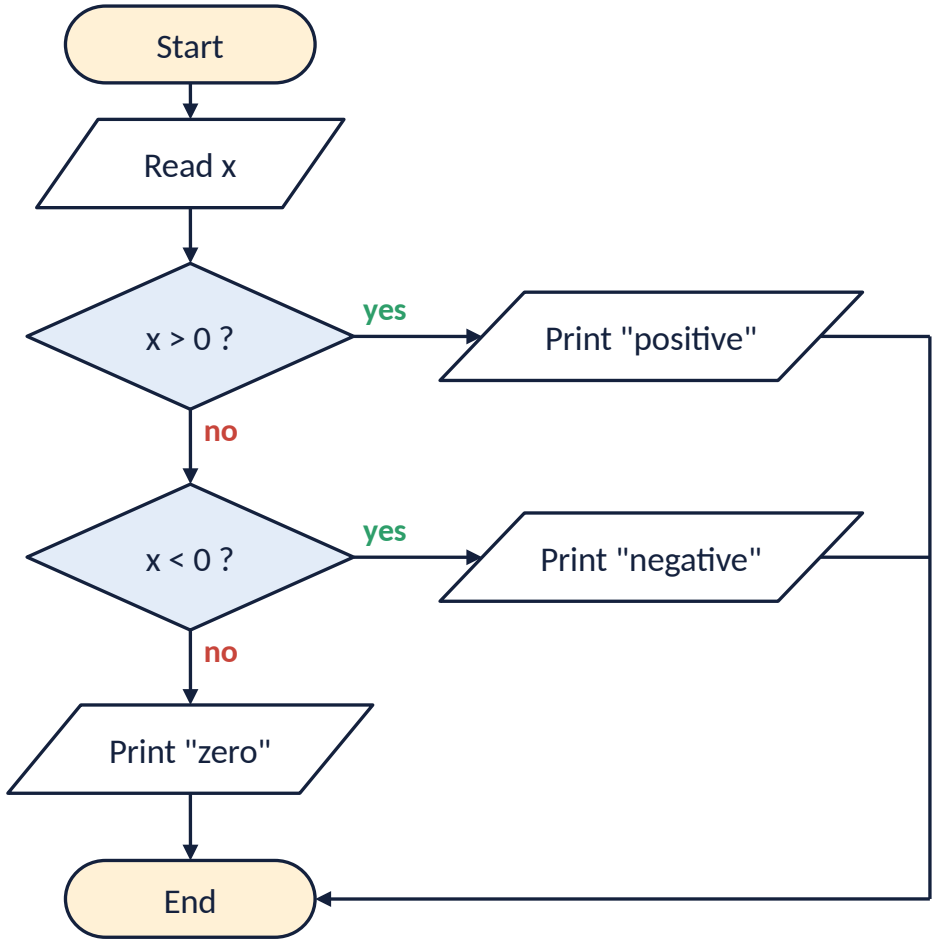
READ / PRINT
→ parallelogram

x ← …
→ rectangle

WHILE cond
→ diamond

END WHILE
→ arrow back up

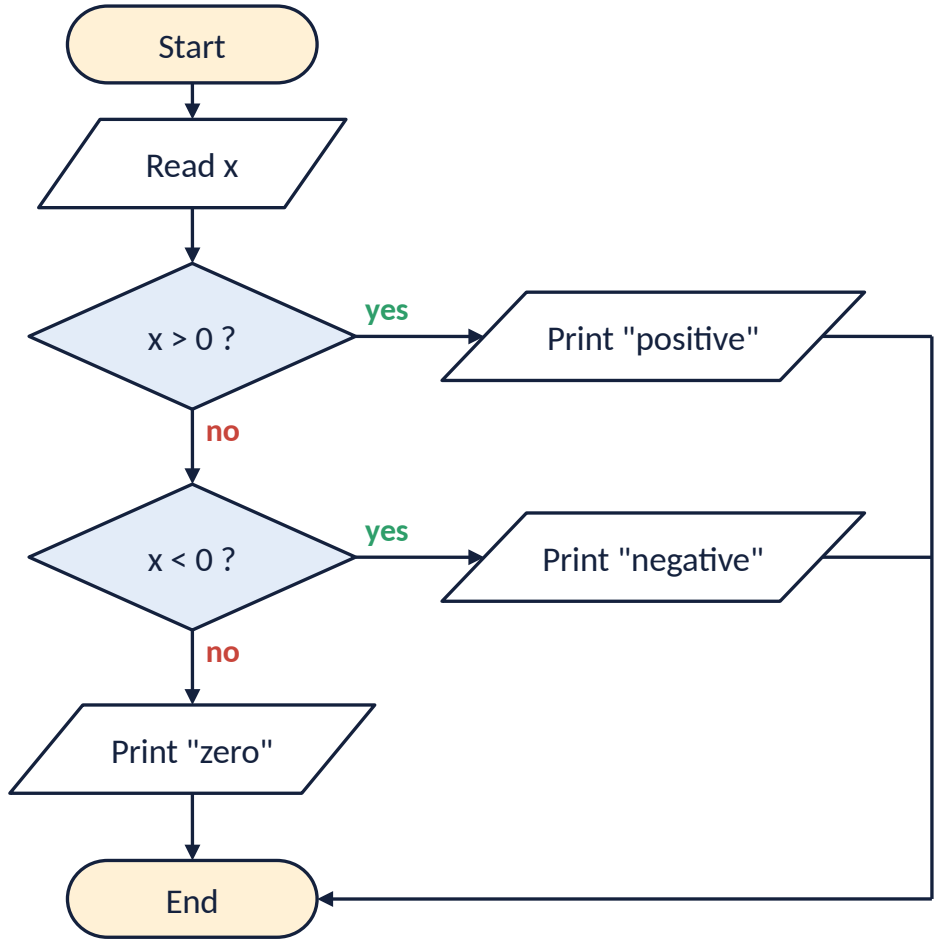
Your Turn: Write the Pseudocode



Task (pairs · 4 min)

1. Write the pseudocode, using IF … THEN … ELSE … END IF.
2. Which kind of structure is this: sequence, selection or repetition? How can you tell?
3. Choose three test inputs so that every box is visited at least once.

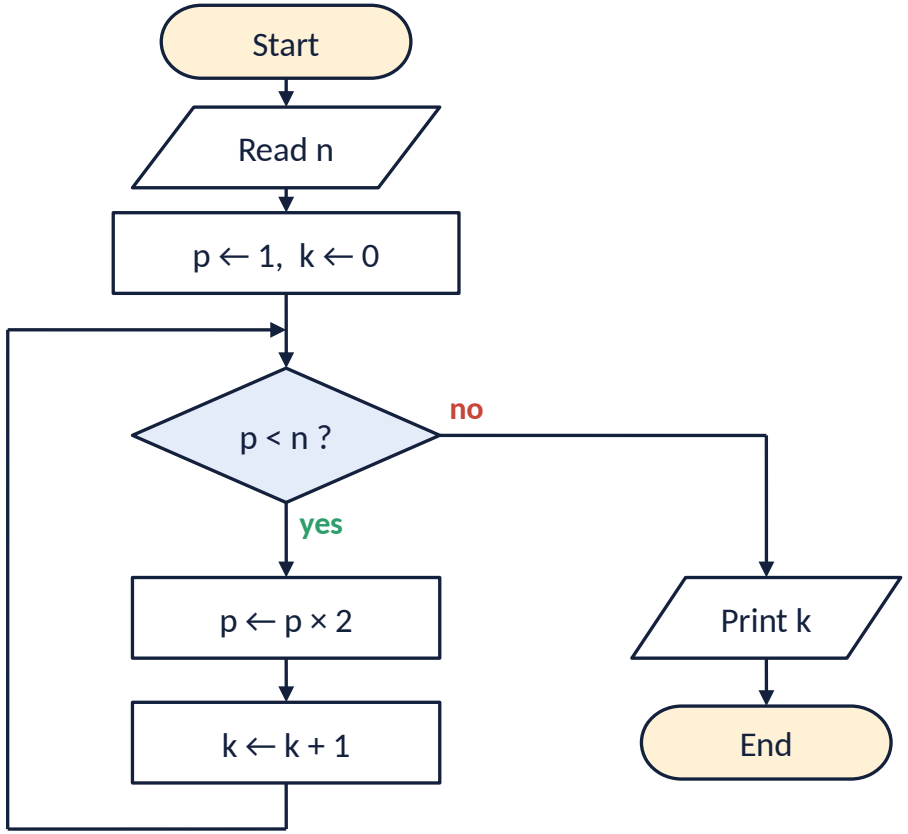
Nested Selection




```
START
  READ x
  IF x > 0 THEN
    PRINT "positive"
  ELSE
    IF x < 0 THEN
      PRINT "negative"
    ELSE
      PRINT "zero"
    END IF
  END IF
END
```

x	path	output
5	yes	positive
−3	no → yes	negative
0	no → no	zero

Tracing a Flowchart with a Trace Table



Trace for n = 20

step	p < n ?	p	k	output
start	—	1	0	
1	yes	2	1	
2	yes	4	2	
3	yes	8	3	
4	yes	16	4	
5	yes	32	5	
exit	**no**	32	5	**5**

What does it compute? Predict the output for n = 1000, then check with a table.

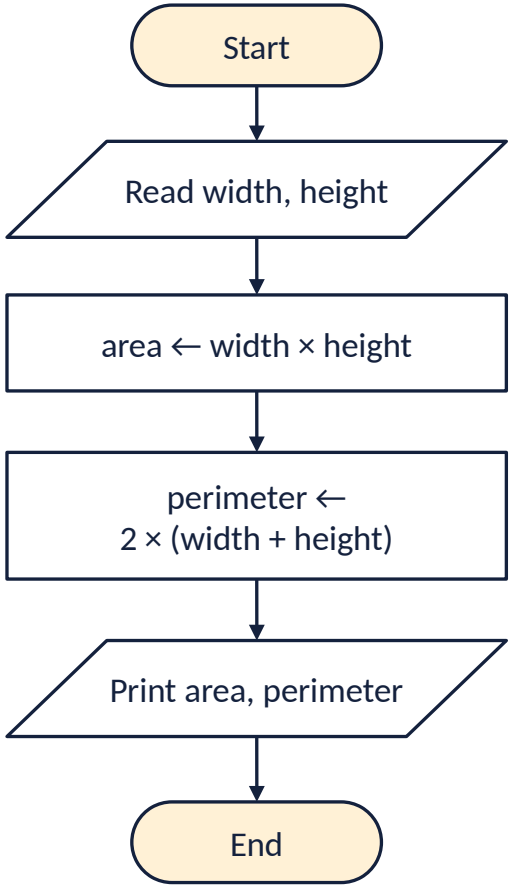


PART 3

Basic Flowchart Examples

Problem → flowchart → C# preview → a real run

Rectangle: Area and Perimeter



code/02_01_RectangleArea/Program.cs

```
Console.Write("Width: ");
double width = double.Parse(Console.ReadLine());    // input
Console.Write("Height: ");
double height = double.Parse(Console.ReadLine());   // input

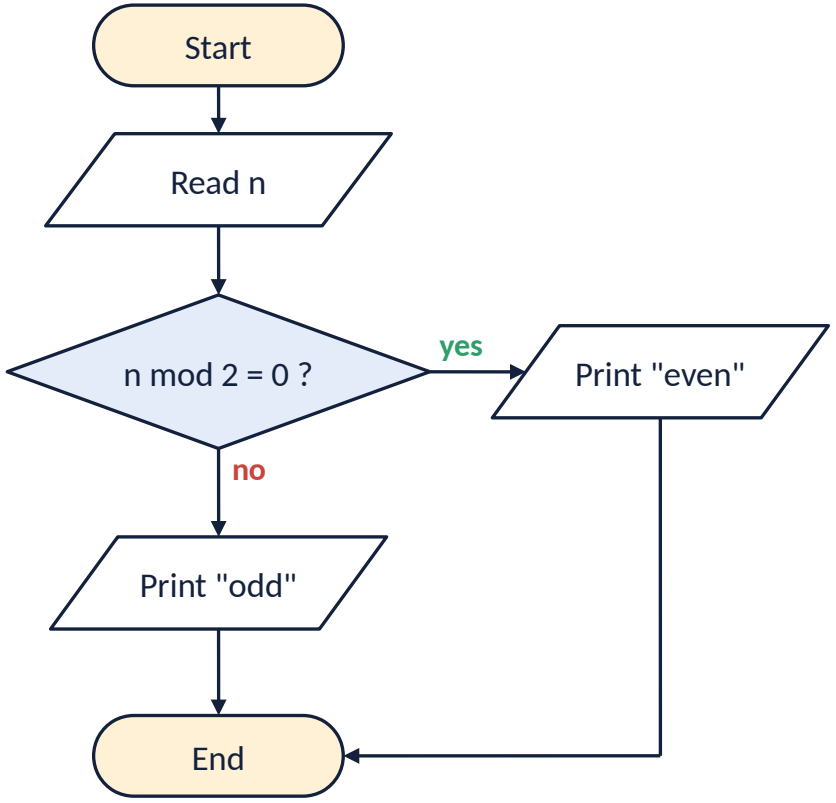
double area = width * height;                       // process
double perimeter = 2 * (width + height);            // process

Console.WriteLine($"Area: {area}");                 // output
Console.WriteLine($"Perimeter: {perimeter}");       // output
```

Console

```
Width: 4.5
Height: 3
Area: 13.5
Perimeter: 15
```

Even or Odd?



code/02_03_EvenOdd/Program.cs

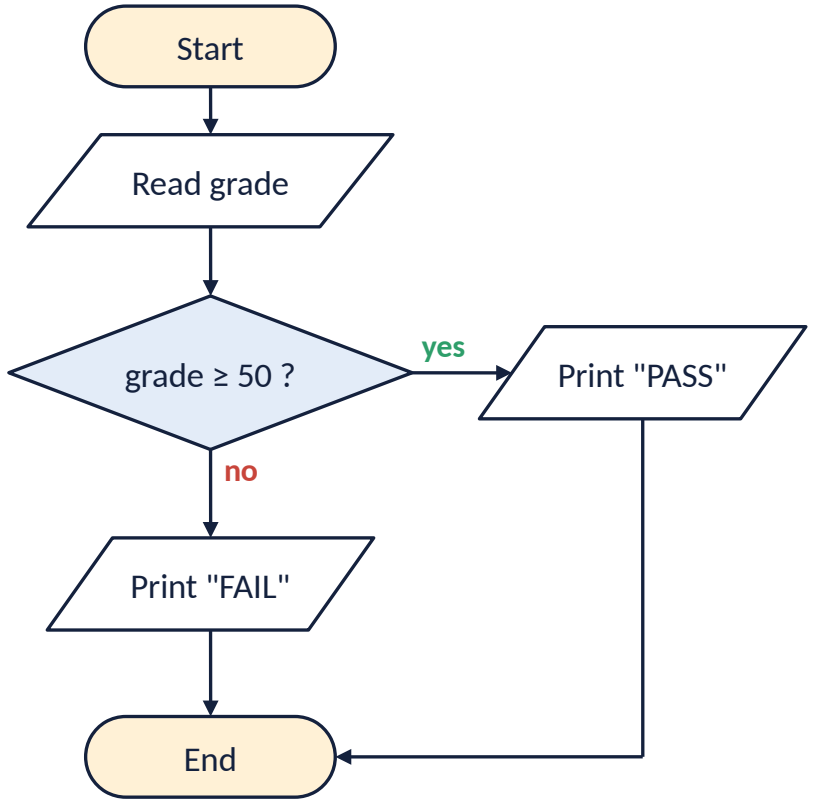
```
Console.Write("Enter an integer: ");
int n = int.Parse(Console.ReadLine());

if (n % 2 == 0)      // % is the remainder operator (mod)
{
    Console.WriteLine($"{n} is even");
}
else
{
    Console.WriteLine($"{n} is odd");
}
```

Console · two runs

```
Enter an integer: 14
14 is even
Enter an integer: -7
-7 is odd
```

Grade → PASS or FAIL



code/02_04_PassFail/Program.cs

```
Console.Write("Grade (0-100): ");
double grade = double.Parse(Console.ReadLine());

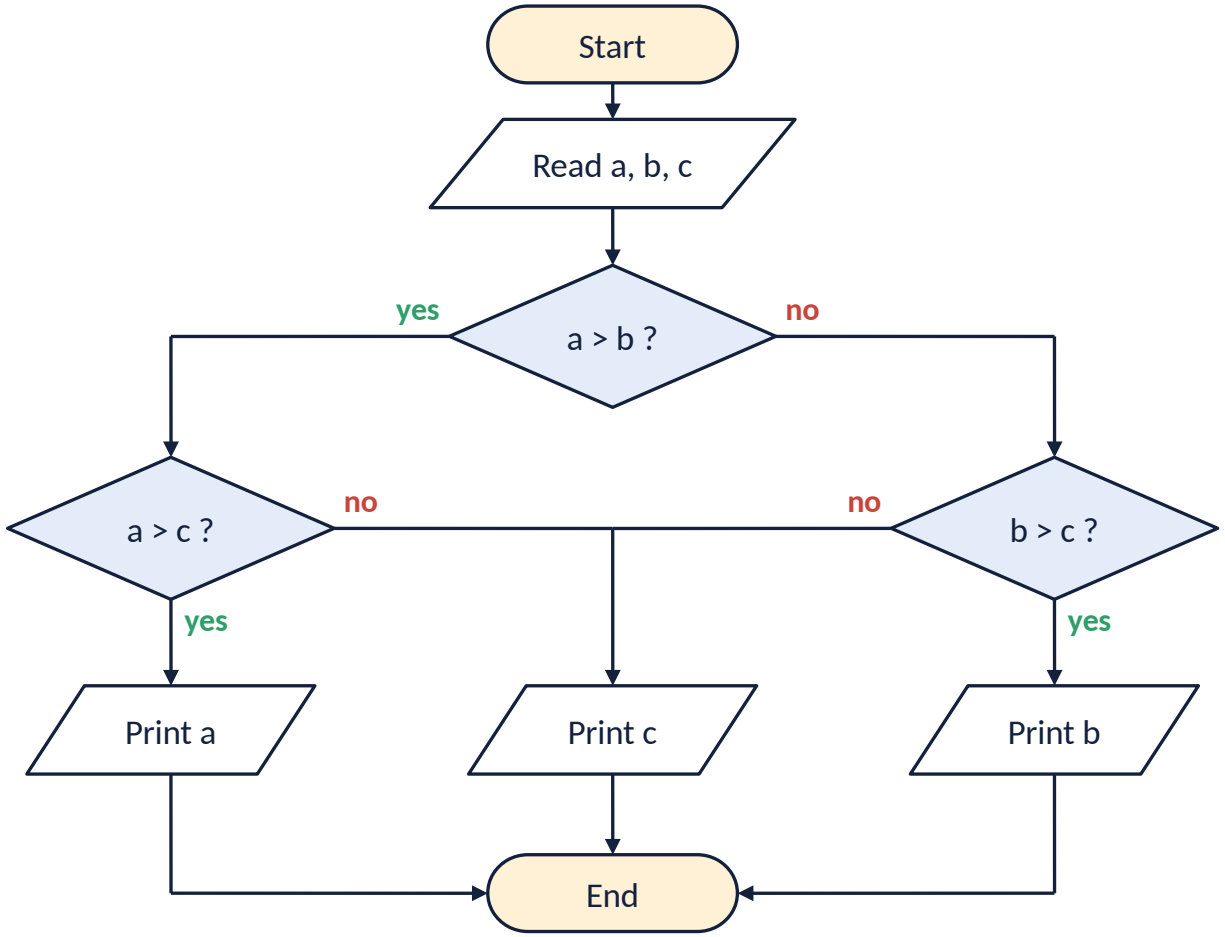
if (grade >= 50)
{
    Console.WriteLine("PASS");
}
else
{
    Console.WriteLine("FAIL");
}
```

Console · two runs

```
Grade (0-100): 49.5
FAIL
Grade (0-100): 50
PASS
```

test grade	output
0	FAIL
49.5	FAIL
50	**PASS**
100	PASS

Maximum of Three: Method 1 (Nested)

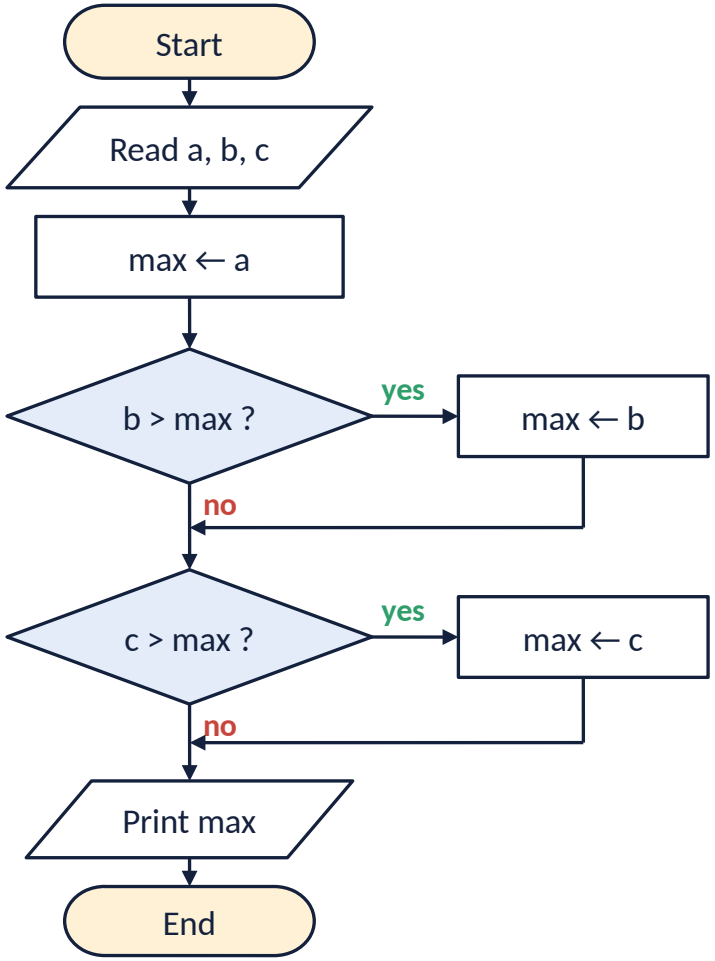


Test every path

a, b, c	decisions	max
4, 9, 7	no → yes	9
8, 3, 8	yes → no	8
2, 5, 9	no → no	9
9, 1, 4	yes → yes	9
−2, −5, −1	yes → no	−1

4 paths → at least 4 tests. Negative numbers must work too.

Maximum of Three: Method 2 (Running Max)



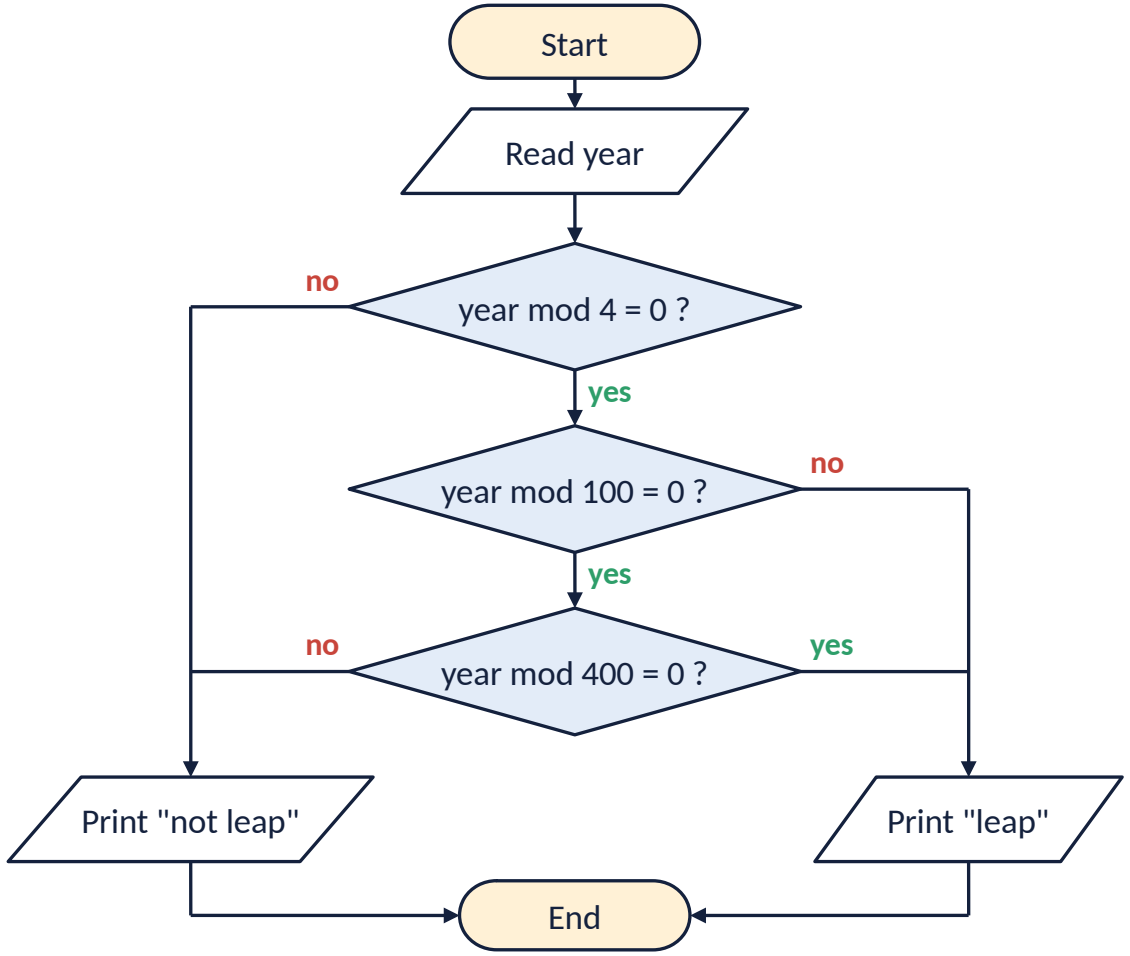
code/02_05_MaxOfThree/Program.cs (Method 2)

```
// Method 2: keep the largest value seen so far
int max = a;
if (b > max)
{
    max = b;
}
if (c > max)
{
    max = c;
}
Console.WriteLine($"Method 2 (running max): {max}");
```

Console

```
a = 4
b = 9
c = 7
Method 1 (nested): 9
Method 2 (running max): 9
```

Leap Year



The same rule as ONE condition

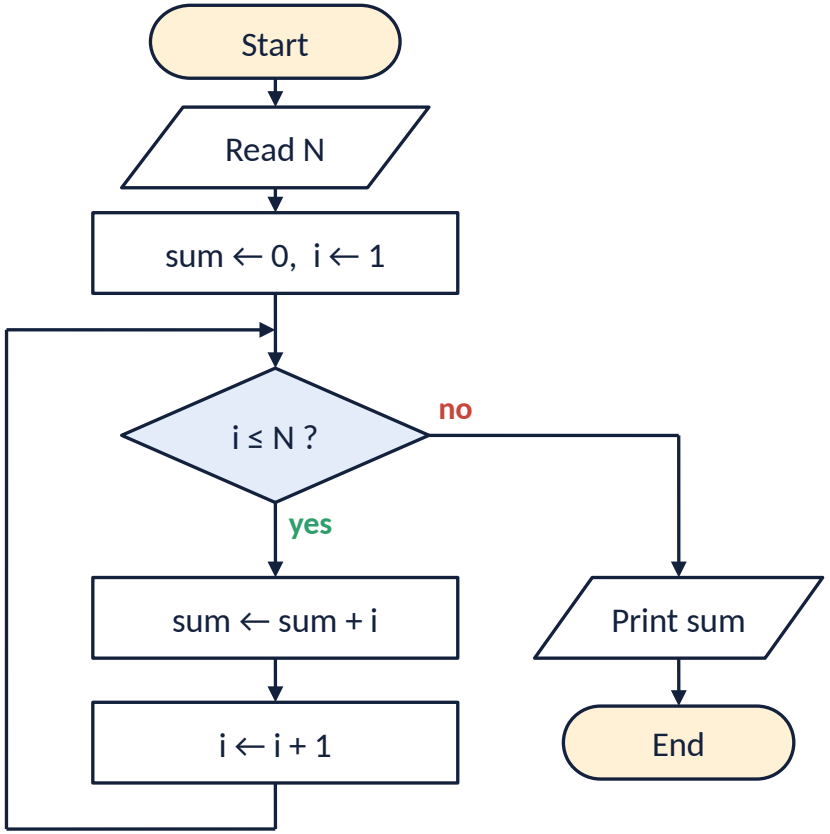
```
IF (year mod 4 = 0 AND year mod 100 ≠ 0)
      OR year mod 400 = 0 THEN …
```

year	mod 4	mod 100	mod 400	leap?
2024	0	24	24	yes
2026	2	26	26	no
1900	0	0	300	no
2000	0	0	0	yes

code/02_06_LeapYear

```
Year: 1900
1900 is not a leap year
Single-condition check: False
```

Sum 1 + 2 + … + N



code/02_07_SumToN/Program.cs

```
Console.Write("N = ");
int n = int.Parse(Console.ReadLine());

int sum = 0;
int i = 1;
while (i <= n)       // decision box: "yes" enters the loop body
{
    sum = sum + i;
    i = i + 1;       // update: without it the loop never ends
}

Console.WriteLine($"Sum 1..{n} = {sum}");
```

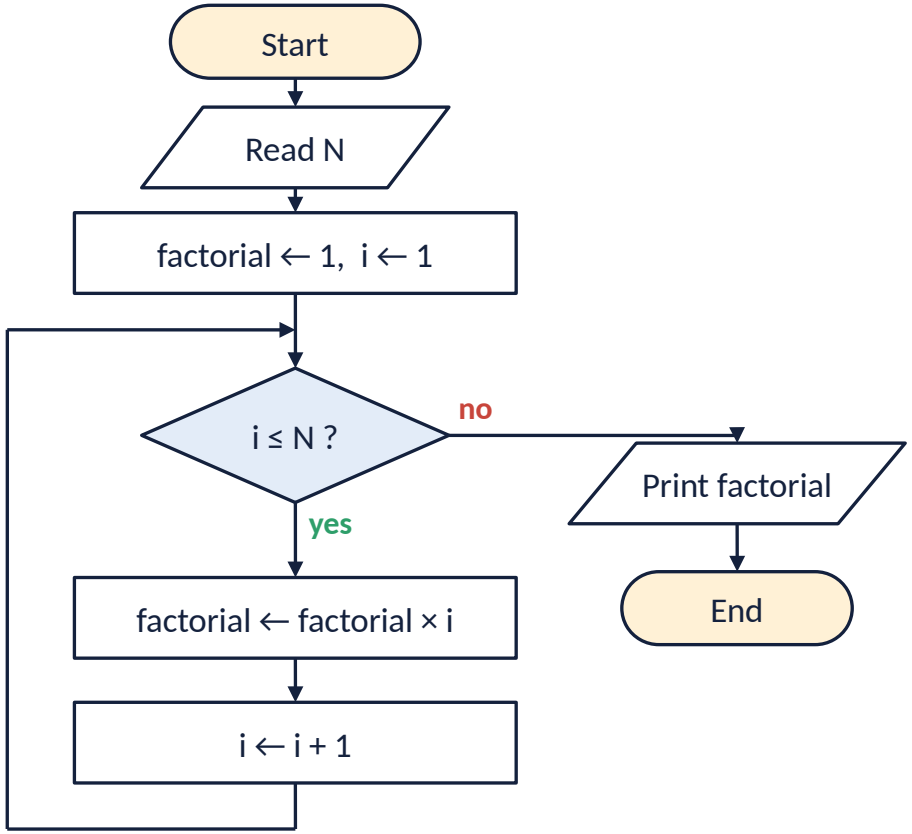
Console

```
N = 100
Sum 1..100 = 5050
```

Console · edge case

```
N = 0
Sum 1..0 = 0
```

Factorial N! = 1 × 2 × … × N



code/02_08_Factorial/Program.cs

```
Console.Write("N = ");
int n = int.Parse(Console.ReadLine());

long factorial = 1;
int i = 1;
while (i <= n)
{
    factorial = factorial * i;
    i = i + 1;
}

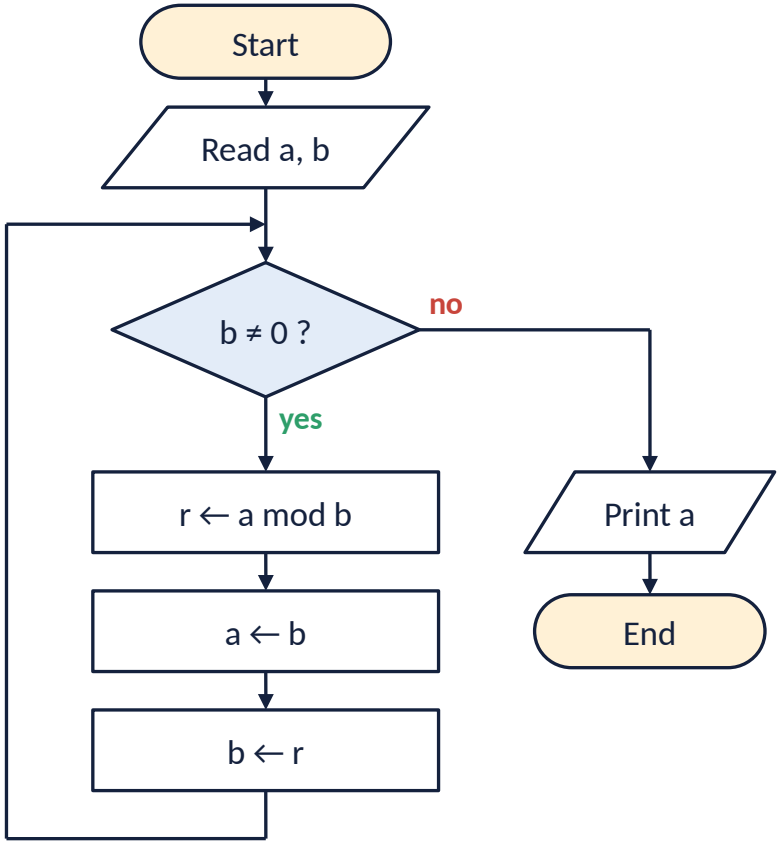
Console.WriteLine($"{n}! = {factorial}");
```

Console · two runs

```
N = 5
5! = 120
N = 20
20! = 2432902008176640000
```

Why long? int stops at 12!; long reaches 20!

Euclid's GCD as a Flowchart




```
Console.Write("a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("b = ");
int b = int.Parse(Console.ReadLine());

while (b != 0)
{
    int r = a % b;   // r <- a mod b
    a = b;
    b = r;
}

Console.WriteLine($"GCD = {a}");
```

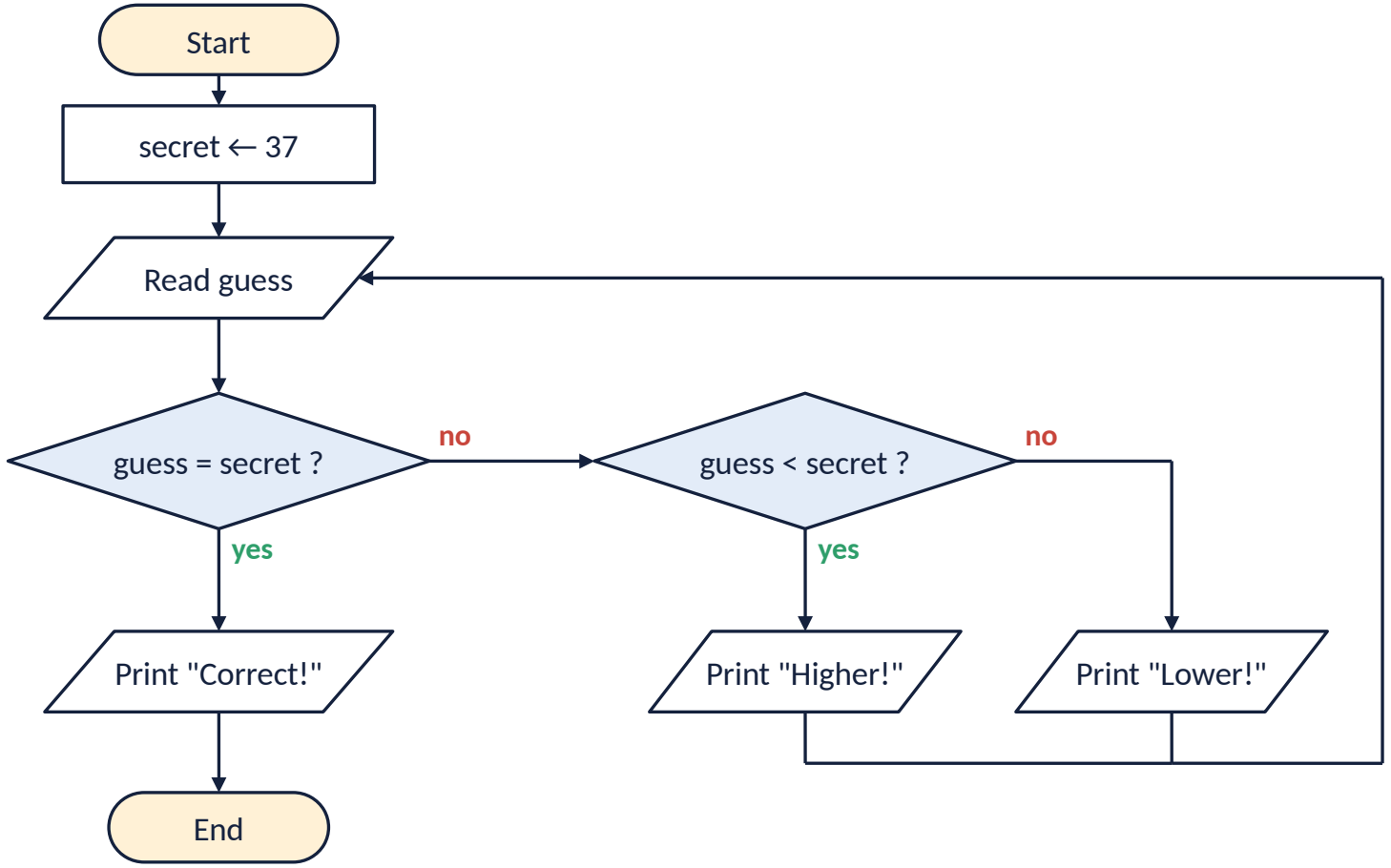
```
Console
a = 48
b = 18
GCD = 6
```

Trace (48, 18)

a	b	b ≠ 0?	r
48	18	yes	12
18	12	yes	6
12	6	yes	0
6	0	no	—

Output: 6
b gets smaller every pass, so the loop must stop.

Guess the Number: the Flowchart



code/02_10_GuessNumber

```
Your guess: 50
Lower!
Your guess: 25
Higher!
Your guess: 37
Correct!
```

Reading the chart

- The upward arrow on the right makes the loop.
- Both hints go back to “Read guess”.
- Only “yes” at the first diamond leaves the loop.

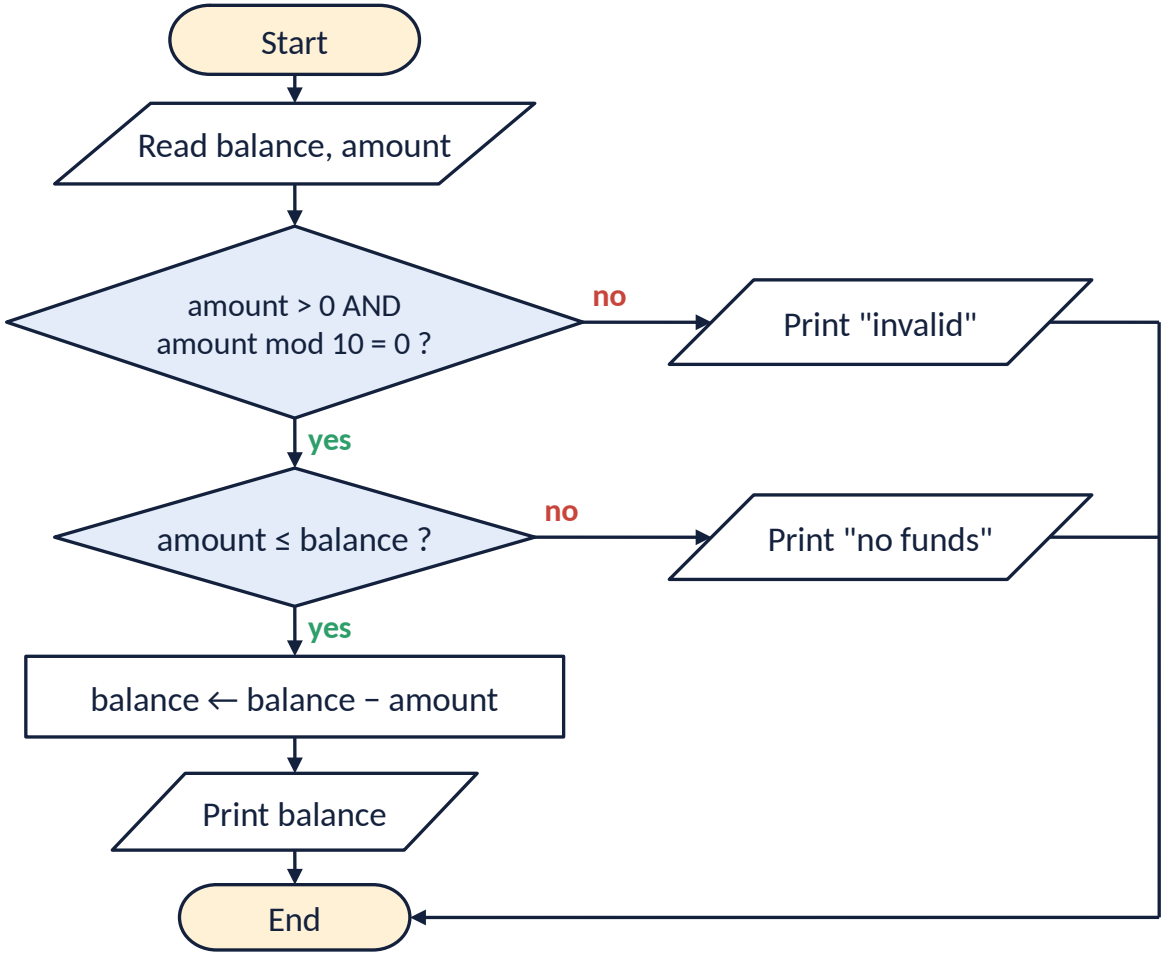
Guess the Number: the Code

code/02_10_GuessNumber/Program.cs

```
 1  int secret = 37;   // fixed for now (random number in Week 7)
 2
 3  Console.Write("Your guess: ");
 4  int guess = int.Parse(Console.ReadLine());
 5
 6  while (guess != secret)
 7  {
 8      if (guess < secret)
 9      {
10          Console.WriteLine("Higher!");
11      }
12      else
13      {
14          Console.WriteLine("Lower!");
15      }
16      Console.Write("Your guess: ");
17      guess = int.Parse(Console.ReadLine());   // read again
18  }
19
20  Console.WriteLine("Correct!");
```

lines 3–4	Read guess (first time)
line 6	diamond guess = secret? — loop runs while NOT equal
lines 8–15	inner diamond: Higher! / Lower!
lines 16–17	Read guess again = the back-edge
line 20	the only way out: Correct!

ATM Withdrawal




```
START
  READ balance, amount
  IF amount > 0 AND amount mod 10 = 0 THEN
    IF amount ≤ balance THEN
      balance ← balance − amount
      PRINT balance
    ELSE
      PRINT "no funds"
    END IF
  ELSE
    PRINT "invalid"
  END IF
END
```

balance, amount	output
500, 200	300
500, 500	0
500, 510	no funds
500, 25	invalid
500, −10	invalid



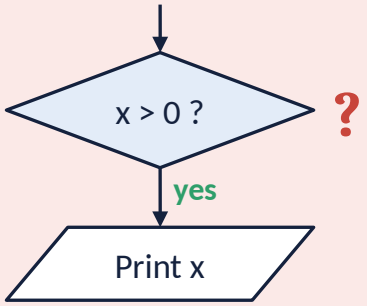
PART 4

Mistakes, Quiz, Tools and Practice

What goes wrong, how to check yourself, and what to draw with

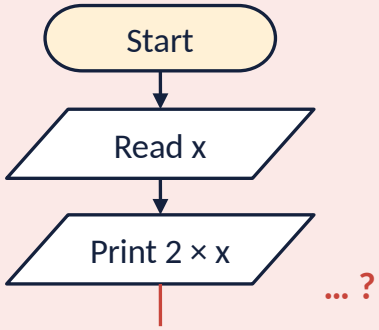
Four Flowchart Bugs (and the Fix)

Decision with one exit



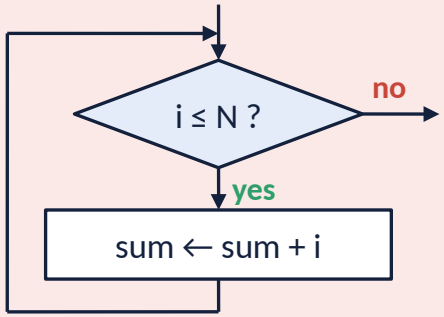
Every diamond needs two labelled exits: add the “no” path.

Missing End



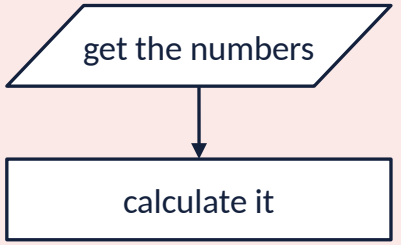
Every path must reach one End terminal.

Loop without update



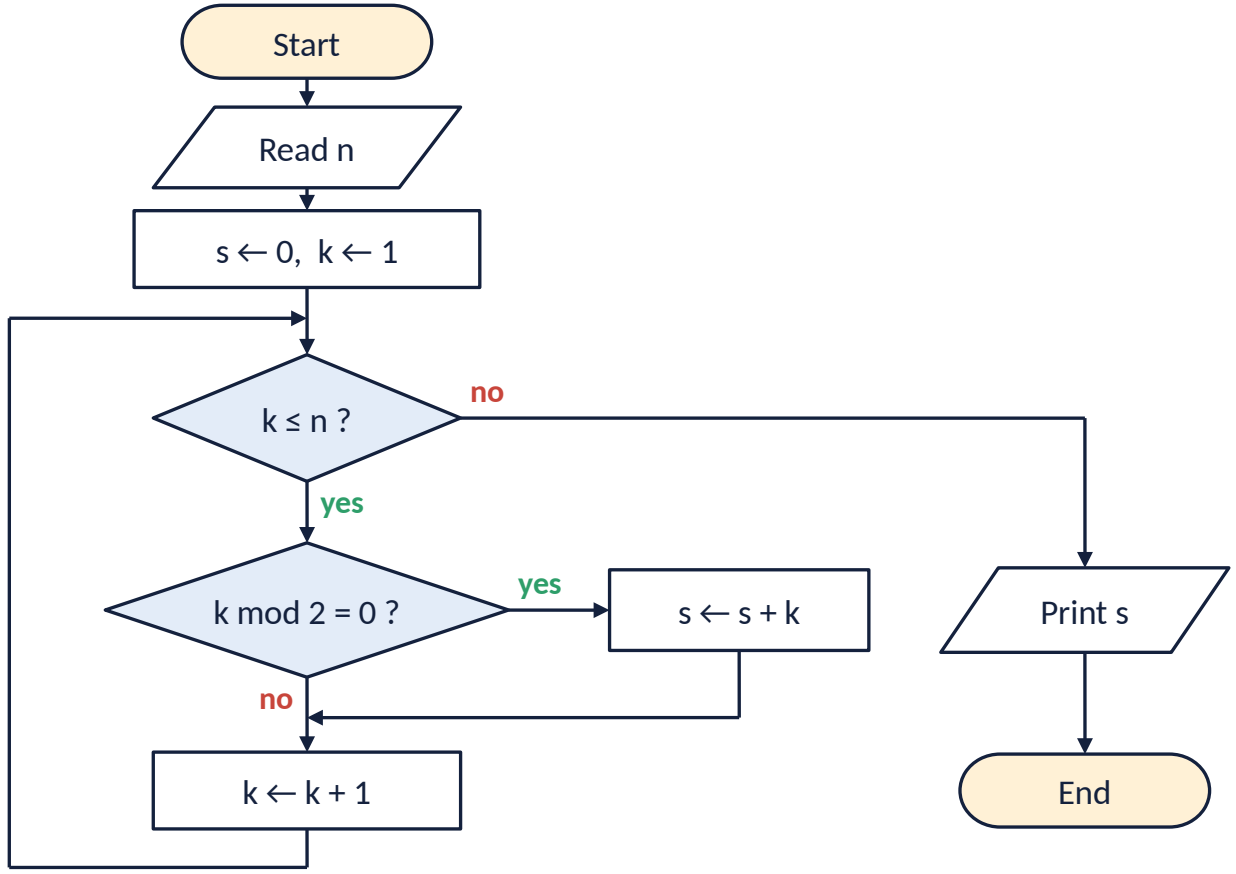
Nothing changes i → infinite loop. Add i ← i + 1.

Ambiguous text



Say exactly what happens: Read a, b · sum ← a + b.

What Does This Flowchart Print for n = 6?



Input: n = 6

A 21

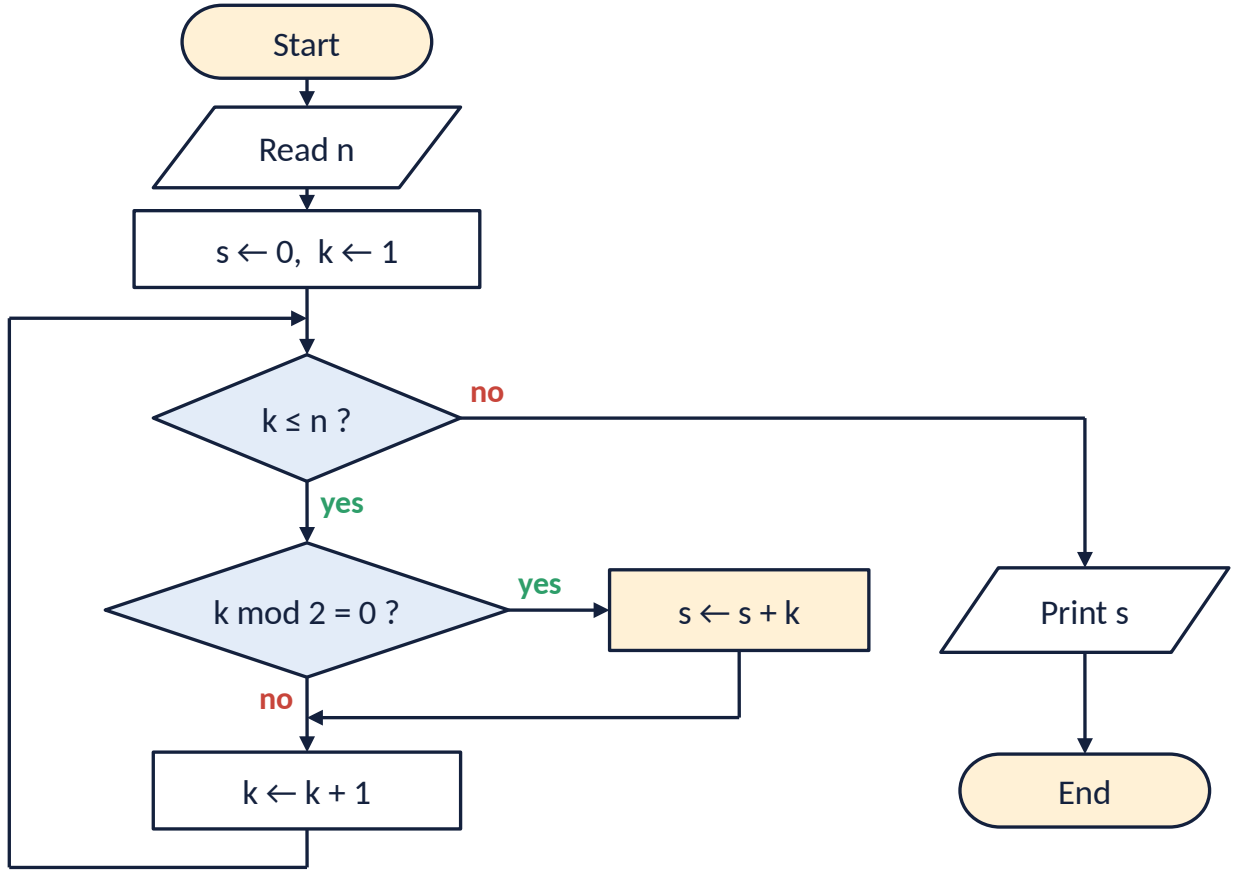
B 12

C 9

D it never stops

Hint: use a trace table with columns k, s.

Answer B: 12 (2 + 4 + 6)



k	k ≤ n ?	k mod 2 = 0 ?	s
1	yes	no	0
2	yes	yes	2
3	yes	no	2
4	yes	yes	6
5	yes	no	6
6	yes	yes	12
7	no	—	Print 12

It adds the even numbers from 1 to n.

What to Draw Flowcharts With

Pen & paper

For exams and quick sketches. Pencil + eraser; leave space between boxes.

draw.io (diagrams.net)

Free, in the browser. Flowchart shape library; export PNG / PDF.

Flowgorithm

Free (Windows). Runs your flowchart step by step and shows it as C# code.

Mermaid

Write text, get a chart. Rendered by GitHub, GitLab and mermaid.live.

Mermaid · absolute value

```
flowchart TD
  A([Start]) --> B[/Read x/]
  B --> C{"x < 0 ?"}
  C -- yes --> D["x ← −x"]
  C -- no --> E[/Print x/]
  D --> E
  E --> F([End])
```

Try it: paste the text into mermaid.live and you get the absolute-value chart.

>_ IN-CLASS · PAIRS · 30 MIN

Your Turn: Draw the Flowchart

A Celsius → Fahrenheit

Read C; print F = C × 9 / 5 + 32.

sequence

B Larger of two

Read a and b; print the larger one (or “equal”).

`selection`

C Ticket price

Age < 12: 50 TL; age ≥ 65: 60 TL; otherwise 100 TL.

`selection`

D Multiplication table

Read n; print n × 1 … n × 10, one per line.

`repetition`

E Average of N numbers

Read N, then N numbers; print their average.

`repetition`

F Guess + counter

Example 9, but also print how many guesses were needed.

`loop + decision`

>_ HOMEWORK · DUE BEFORE WEEK 3

Practice + a First Look at the Code

On paper

- H1 · Draw flowcharts for Week 1 exercises A (even/odd), B (leap year, your own layout) and F (Celsius → Fahrenheit).
- H2 · Trace the GCD flowchart for (84, 36) and (17, 5) in trace tables.
- H3 · Extend the guess-the-number chart with a guess counter.
- Post photos of your pages to the course group.

On the computer

- H4 · Run two preview programs from the code folder; compare each run with its flowchart.
- Optional: rebuild the Sum 1..N chart in Flowgorithm and run it.

```
> cd code/02_07_SumToN
> dotnet run
N = 10
Sum 1..10 = 55
```

>_ SUMMARY

Key Takeaways

01 A flowchart is a picture of an algorithm: shapes = kinds of steps, arrows = order.

02 Six symbols: terminal, input/output, process, decision, predefined process, connector.

03 Rules: one Start, one End; two labelled exits per decision; arrows everywhere; no crossings.

04 Sequence, selection and repetition each have a pattern; an upward arrow means a loop.

05 Convert both ways: one box ≈ one pseudocode line; diamond + back-edge = WHILE.

06 Trace tables test a flowchart before any code exists.

NEXT WEEK

Programming languages, variables, data types, constants

>_

MCMP113
Course materials